

Encoder-Decoder Models for Grapheme-to-Phoneme Conversion

Catherine Li

May 2026

1 Implementation

The Dataset, model and various model components are implemented as classes, while training and helper functionalities are mostly implemented as functions.

1.1 G2PDataset

This class first pads the target phonemes with <SOS>, and <EOS>; then the class maps each item in the word and phoneme sequence to their index representation. Additionally, the class truncates the character and phoneme sequences so the sequence length is equivalent to *max_len*.

1.2 collate_fn

a custom *collate_fn* was used to pad all source and target sequences to the same length with <PAD>= 0

1.3 LSTMCellWithContext

This class is used in the LSTMModelWithContext class. This is mostly based on the LSTM tutorial, except the *bool* variable *with_context* is introduced. When true, the LSTMCellWithContext initialises the context weights, *V*, and includes context-related operations during the forward pass. If false, it works the same as in the tutorial.

1.4 Encoder-Decoder Implementation

1.4.1 LSTMModelWithContext

This is used in both the encoder and decoder classes. Building on the LSTMModel from the tutorial, it additionally includes a method for cross-attention (CA) calculations, feeding the fixed-context vector and feeding the context vector with CA.

The *init* method evaluates the *with_context* variable with *_context* for LSTMCellWithContext and initialises the cells with the assumption that the context vectors are only passed to the first layer of cells when *with_context* is true.

In the forward method, the *fixed_context* is added to account for the change of the *hidden* variable each time the forward is called during autoregressive generation in Section 1.4.3.

CA is implemented as a method as there are no class variables that it needs to store, like the other classes. *torch.bmm* is used to better take advantage of the gpu. The calculated attention weights are formatted with *torch.stack* and then returned for the Attention Analysis in Section 2.4. A mask is created and applied before softmax so the model does not pay attention to padding positions in the encoder. na

1.4.2 Encoder and Decoder

Both classes are based on the *LSTMClassifier* from the LSTM tutorial, and act as wrappers to pass the input as embeddings to the *LSTMModelWithContext* as well as calculate the logits with a linear layer. The Decoder additionally returns the attention weights and the forward method has *fixed_context* in its method signature.

1.4.3 Seq2Seq Encoder-Decoder

This class is a wrapper that handles the encoder-decoder interactions, and implements teacher forcing as well as autoregressive generation.

The helper method *_autoregressive_generation* implements the autoregressive generation by first feeding the decoder a *decoder_input* a batch of seeds, then it

1. generates the output logits as well as predictions
2. calls the decoder again and only feeds in the last prediction as input, along with the decoder hidden state. Hence, the *fixed_context* variable is needed for the *LSTMModelWithContext* in Section 1.4.1
3. repeats from step 1

The tensor *finished* keeps track of which sequence in the batch has not generated <EOS>.

The *forward* method can either have teacher forcing or autoregressive generation with the target sequence length leaked to the autoregressive generation.

The *greedy_decode* runs the encoder-decoder in an auto-regressive fashion without leaking any target phoneme sequence information.

1.5 Hyperparameter Tuning

The hyperparameter tuning is done with grid search using the suggested range of values. The results are presented in Table 1, where final values and the best performing epoch’s metrics are presented. The best epoch is determined by the best validation Phoneme Error Rate (PER) value of all the epochs for a particular configuration.

1.5.1 Observations

The table is in ascending order of the best epoch’s validation PER. In Table 1, the results exhibit a descending order for the learning rate. Within each learning rate category (though specifically 0.001), hidden state sizes are roughly descending; and within each category of hidden sizes, the embedding dimensions are also roughly descending. This reflects the trend within the values explored that greater hyperparameter values are more effective, with learning rates being the most impactful and embedding dimensions the least, relatively. The greater learning rate needed may be connected to the chosen batch size, 256, where it is safer (i.e. the results are more stable) when taking bigger steps during gradient descent as the derived gradient would have a smaller variance, and, thus, less noise. Another reason is that the model is being updated less due to the larger batch size, so within a limited epoch range, the larger learning rate performs better. A larger hidden size may be more appropriate as it captures more information about the input word in the encoder, and the phoneme sequence (whether autoregressively generated or taken from ground truth). Lastly, the larger embedding dimension may capture more features in the latent space for the model to use.

The top row of Table 1 reflects the best performing configuration for the no-context Encoder-Decoder model in terms of validation PER. Hence, its configuration is chosen for evaluating the three model variants. The chosen configuration is as follows:

- Learning Rate: 0.001
- Embedding Size: 128
- Hidden State Size: 256

2 Analysis

All models are trained using early stopping with the patience set to 5, the hyperparameters specified in Section 1.5, the random seed set to 42, and the batch size set to 256. The loss curves are displayed in Figure 1, and the test-set results are reported in Table 2.

2.1 Convergence and Loss

The model with CA-context converged the slowest, at epoch 20, while the with fixed context converges the fastest at epoch 15, displayed in Table 2. Additionally, the model with CA-context performs the best across all three metrics, which includes having lowest test loss.

Table 1: Hyperparameter tuning and configurations, in ascending order of the best epoch’s PER

config no.	lr	embed dim	hidden size	final loss	val acc	final PER	val PER	best epoch	best epoch’s PER	best epoch’s acc	best epoch’s loss
26	0.00100	128	256	1.073149	0.496581	0.160554		5	0.160554	0.496581	1.073149
23	0.00100	64	256	1.142084	0.468709	0.171905		5	0.171905	0.468709	1.142084
20	0.00100	32	256	1.137553	0.446897	0.180637		5	0.180637	0.446897	1.137553
25	0.00100	128	128	1.538877	0.356617	0.238089		5	0.238089	0.356617	1.538877
22	0.00100	64	128	1.570508	0.343547	0.245629		5	0.245629	0.343547	1.570508
19	0.00100	32	128	1.651673	0.316368	0.264137		5	0.264137	0.316368	1.651673
24	0.00100	128	64	2.156234	0.207738	0.351428		5	0.351428	0.207738	2.156234
21	0.00100	64	64	2.263726	0.178049	0.385458		5	0.385458	0.178049	2.263726
18	0.00100	32	64	2.302296	0.139617	0.412716		5	0.412716	0.139617	2.302296
17	0.00010	128	256	2.439334	0.103350	0.446929		5	0.446929	0.103350	2.439334
14	0.00010	64	256	2.681169	0.045962	0.545014		5	0.545014	0.045962	2.681169
16	0.00010	128	128	2.902485	0.023024	0.610043		5	0.610043	0.023024	2.902485
11	0.00010	32	256	2.793082	0.015321	0.626140		5	0.626140	0.015321	2.793082
13	0.00010	64	128	3.033481	0.004328	0.729821		5	0.729821	0.004328	3.033481
15	0.00010	128	64	3.182580	0.000346	0.805241		5	0.805241	0.000346	3.182580
10	0.00010	32	128	3.164112	0.000606	0.818294		5	0.818294	0.000606	3.164112
12	0.00010	64	64	3.245011	0.000173	0.858932		5	0.858932	0.000173	3.245011
7	0.00001	128	128	3.435995	0.000000	0.862434		5	0.862434	0.000000	3.435995
8	0.00001	128	256	3.274885	0.000000	0.871743		5	0.871743	0.000000	3.274885
5	0.00001	64	256	3.244872	0.000000	0.881925		1	0.872751	0.000000	4.201526
9	0.00010	32	64	3.273589	0.000000	0.893129		1	0.876781	0.000000	3.713562
4	0.00001	64	128	3.685559	0.000000	0.882753		5	0.882753	0.000000	3.685559
2	0.00001	32	256	3.510935	0.000000	0.884405		5	0.884405	0.000000	3.510935
1	0.00001	32	128	3.819561	0.000000	0.890304		4	0.887119	0.000000	3.937859
6	0.00001	128	64	3.978759	0.000000	0.927512		5	0.927512	0.000000	3.978759
3	0.00001	64	64	4.002318	0.000000	0.948713		5	0.948713	0.000000	4.002318
0	0.00001	32	64	4.172225	0.000000	0.967415		4	0.963017	0.000000	4.222859

Table 2: Model Comparison Results on Test Set

model	test loss	test acc	test PER	stopping_epoch
none	1.096418	0.571317	0.129680	17
fixed	1.082816	0.557296	0.133527	15
attn	0.881183	0.586723	0.117885	20

2.2 Architectural Comparison

The fixed-context model performs the worst overall. Particularly, it underperforms relative to Setup 1, the no-context model when evaluated on the test set by $\tilde{0.004}$, $\tilde{0.01}$, and $\tilde{0.004}$ for loss, word accuracy, and PER, respectively. On the other hand, the CA-context model outperforms the no-context model by $\tilde{0.215}$, $\tilde{0.015}$, and $\tilde{0.012}$ in terms of loss, word accuracy, and PER, respectively. These results may suggest that the source word information retained in the fixed context vector may have helped the model to converge faster, but does not necessarily improve prediction results. However, applying CA to the context builds more immediately useful context vectors for the model to leverage. In other words, CA makes the decoding process with context vectors more efficient and accurate.

2.3 Error Analysis

Some examples of errors made by the model on words extracted from the test set are displayed in Figure 2. The no-context model seems to struggle with the hidden-state bottleneck, where performance often degrades towards the end. Specifically, the words that only the no-context failed at has seemingly random predictions such as "UW2" and its unique failures happen at relatively shorter words. In contrast, the mistakes unique to fixed-context models are longer words and more "sensibly" such as having "IH0" instead of "AH0" in "turrets", or pronouncing the past tense suffix "ed" as "AH0 D" instead of "T" in "approached". The CA-context model’s unique mistakes are with

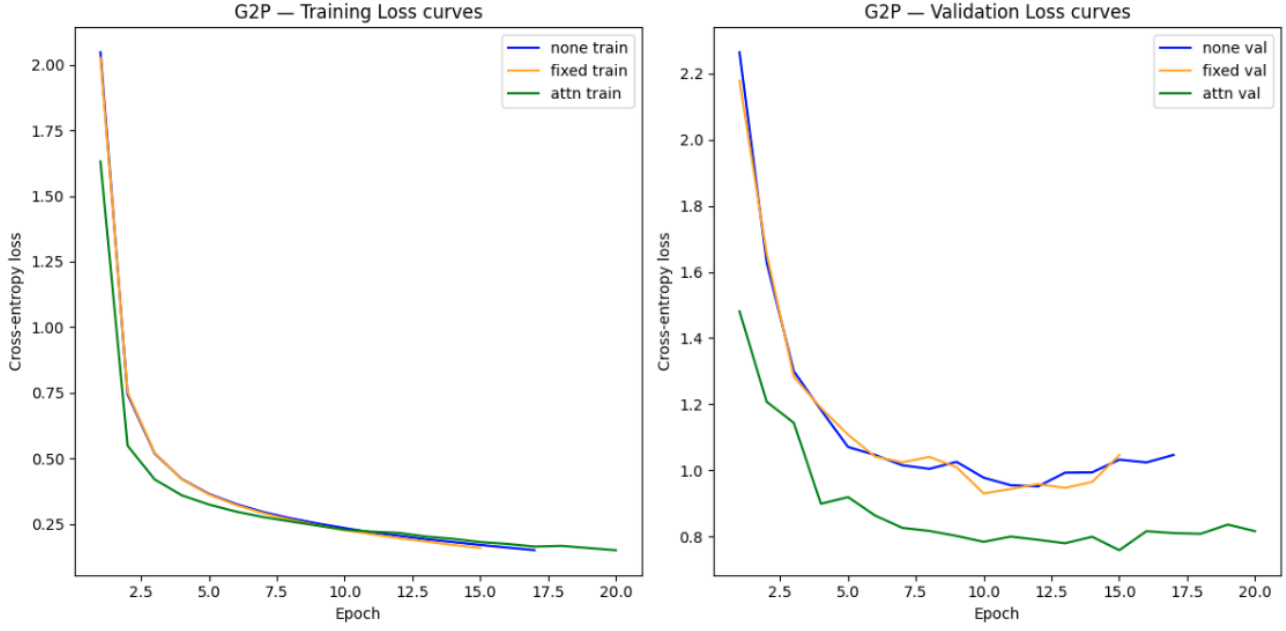


Figure 1: Loss Curve of Training and Validation

only none fails (7 words) Word : lawnmower (len=9) Target : L A01 N M OW0 ER0 Pred attn : L A01 N M OW0 ER0 Pred fixed: L A01 N M OW0 ER0 Pred none : L A01 N M OW0 R UW2 Word : smoothest (len=9) Target : S M UW1 DH AH0 S T Pred attn : S M UW1 DH AH0 S T Pred fixed: S M UW1 DH AH0 S T Pred none : S M UW1 DH T S UW2 Word : mcmonagle (len=9) Target : M AH0 K M AA1 N AH0 G AH0 L Pred attn : M AH0 K M AA1 N AH0 G AH0 L Pred fixed: M AH0 K M AA1 N AH0 G AH0 L Pred none : M AH0 K AA1 M AH0 N AH0 L	only fixed fails (3 words) Word : stereotyped (len=11) Target : S T EH1 R IY0 AH0 T AY2 P T Pred attn : S T EH1 R IY0 AH0 T AY2 P T Pred fixed: S T EH1 R IY0 OW0 T AY2 P T Pred none : S T EH1 R IY0 AH0 T AY2 P T Word : approached (len=10) Target : AH0 P R OW1 CH T Pred attn : AH0 P R OW1 CH T Pred fixed: AH0 P R OW1 CH AH0 D Pred none : AH0 P R OW1 CH T Word : turrets (len=7) Target : T ER1 AH0 T S Pred attn : T ER1 AH0 T S Pred fixed: T ER1 IH0 S T Pred none : T ER1 AH0 T S	only attn fails (49 words) Word : decommissioned (len=14) Target : D IY0 K AH0 M IH1 SH AH0 N D Pred attn : D IY2 K AA2 M IH0 S AA1 N AH0 S Pred fixed: D IY0 K AH0 M IH1 SH AH0 N D Pred none : D IY0 K AH0 M IH1 SH AH0 N D Word : undertaker (len=10) Target : AH1 N D ER0 T EY2 K ER0 Pred attn : AH1 N D ER0 T AE2 K ER0 Pred fixed: AH1 N D ER0 T EY2 K ER0 Pred none : AH1 N D ER0 T EY2 K ER0 Word : resistance (len=10) Target : R IH0 Z IH1 S T AH0 N S Pred attn : R EH1 Z IH0 S T AH0 N S Pred fixed: R IH0 Z IH1 S T AH0 N S Pred none : R IH0 Z IH1 S T AH0 N S	all fail (132 words) Word : eccentricities (len=14) Target : EH2 K S EH0 N T R IH1 S IH0 T IY0 Z Pred attn : EH2 S AH0 N T R IH1 S AH0 T IY0 Z Pred fixed: EH0 K S EH2 N T R IH1 S IH0 T IY0 Z Pred none : EH2 K S AH0 N T R IH1 S AH0 T IY0 Z Word : photosynthesis (len=14) Target : F OW2 T OW0 S IH1 N TH AH0 S IH0 S Pred attn : F OW1 T OW0 S IH2 N TH AH0 S IH0 S Pred fixed: F OW2 T OW0 S IH1 N TH IY0 AH0 S T S Pred none : F OW2 T AH0 Z IH1 N TH S AH0 N Word : vulgarization (len=13) Target : V AH2 L G ER0 IH0 Z EY1 SH AH0 N Pred attn : V L AH0 G ER0 AH0 Z EY1 SH AH0 N Pred fixed: V AH2 L G R IH0 Z EY1 R IH0 T AH0 N Pred none : V AH2 L G ER0 AH0 Z EY1 SH AH0 N
---	---	---	--

Figure 2: Example errors made by the different models

respect to the numerical stress markers and vowel sounds, also on longer words like the fixed-context model. The CA-model also seemingly maintains the position of the consonant pronunciations better.

Additionally, the intersection between all the mistakes is generally happens longer words, where performance degrades in a mixture of ways (such as the ones described above). Interestingly, the CA-context model seems to have predicted the most inaccurately. This result may be ascribed to the small size of the sample extracted from the test set.

2.4 Attention Analysis

The attention analysis is visualised in Figure 3. The longer words have the attention weights mostly appearing diagonally downwards, from left to right.

The first prediction generally only focuses on the last letter (and its hidden state). This makes sense as the initialised decoder hidden state at time zero is essentially the same as the final encoder hidden state.

Additionally, the CA focuses on the encoder hidden states 1 or 2 letters around the characters responsible for the pronunciation. This trend reveals that the model mostly attends to characters most relevant to each output phoneme, as pronunciations mostly depend on the local letters around a specific letter. To more efficiently obtain information, the attention mechanism may be more inclined to pay attention to the encoder hidden states of the set of letters the model is intending to pronounce. For example, in the "conservationists" graph, before predicting "SH", it pays attention to the "i", "o", and "n" hidden state (which is where their respective information is newly inputted

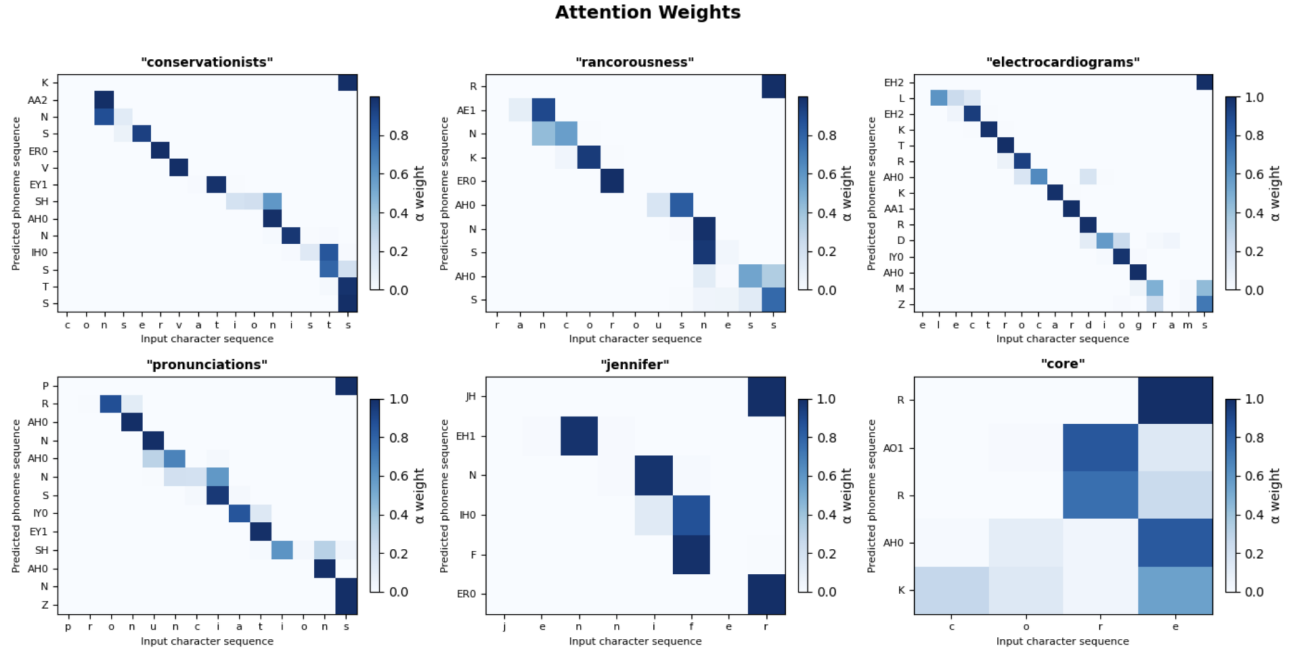


Figure 3: Heat map of cross attention weights. The title is the word analysed, from the test set.

to the hidden state) as the model potentially needs more information for identifying the "SH" pronunciation in the "tion" pattern.

Additionally, the attention weights of "core" appear the most diffuse. This could be an artifact of the shortness of the word, and perhaps more importantly, the complex vowel and consonant interactions, where 'c' has multiple pronunciations, the 'e' silent and the letters 'or' merge together.